

Spark Stream Processing

www.3aayaam.in

Agenda 1

1. Structured Streaming
2. Stream Processing Model
3. Trigger
4. Output Modes
5. PySpark Implementation

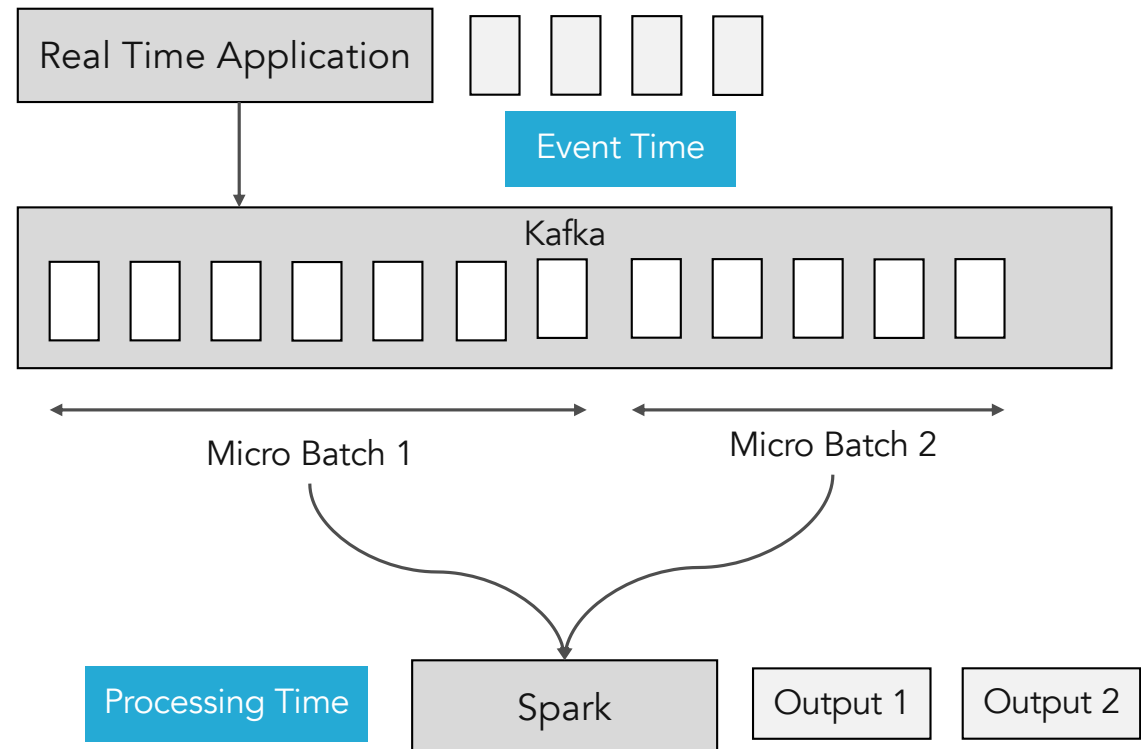
Streaming Ingestion

- Data from streaming sources.
- Real-time applications, IoT devices, Database changes (CDC).
 - Stock market
 - Financial fraud detection
 - Predictive maintenance
 - Autonomous cars
 - Traffic management
 - Network and log monitoring
 - Predictive maintenance
- Data is ingested in Kafka.



Spark Structured Streaming

- DataFrame API, Spark SQL.
- Micro Batch processing.
- Incremental data processing.
- Event Time – When data is generated.
- Processing Time – When data is processed.



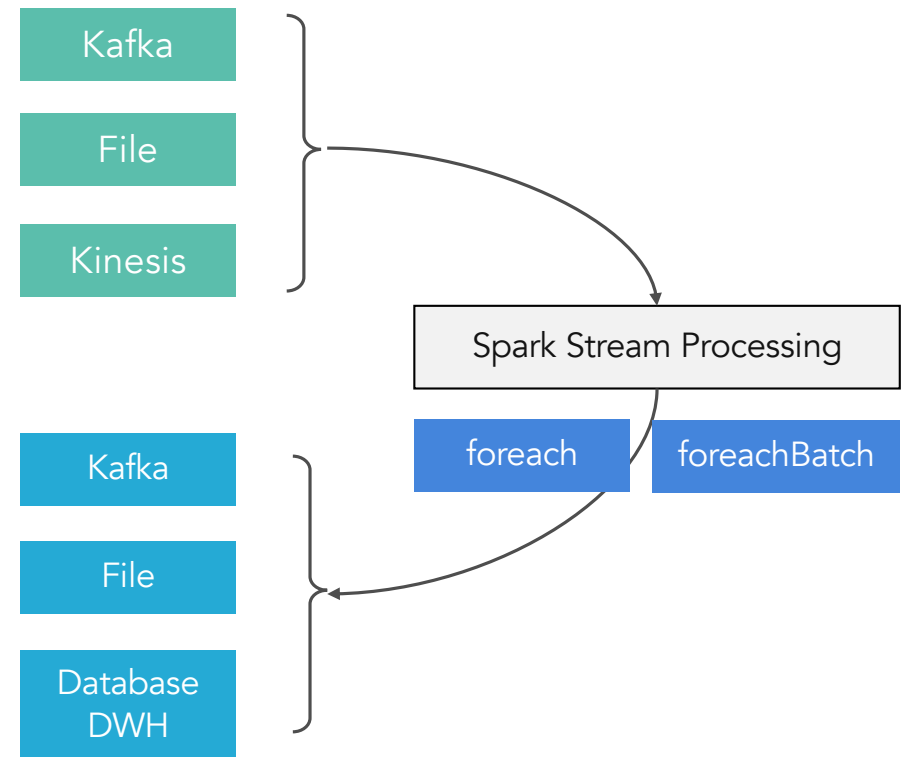
Stream Processing Model

- Data Sources

- Kafka source
- File source
- Socket source
- AWS Kinesis

- Data Sinks

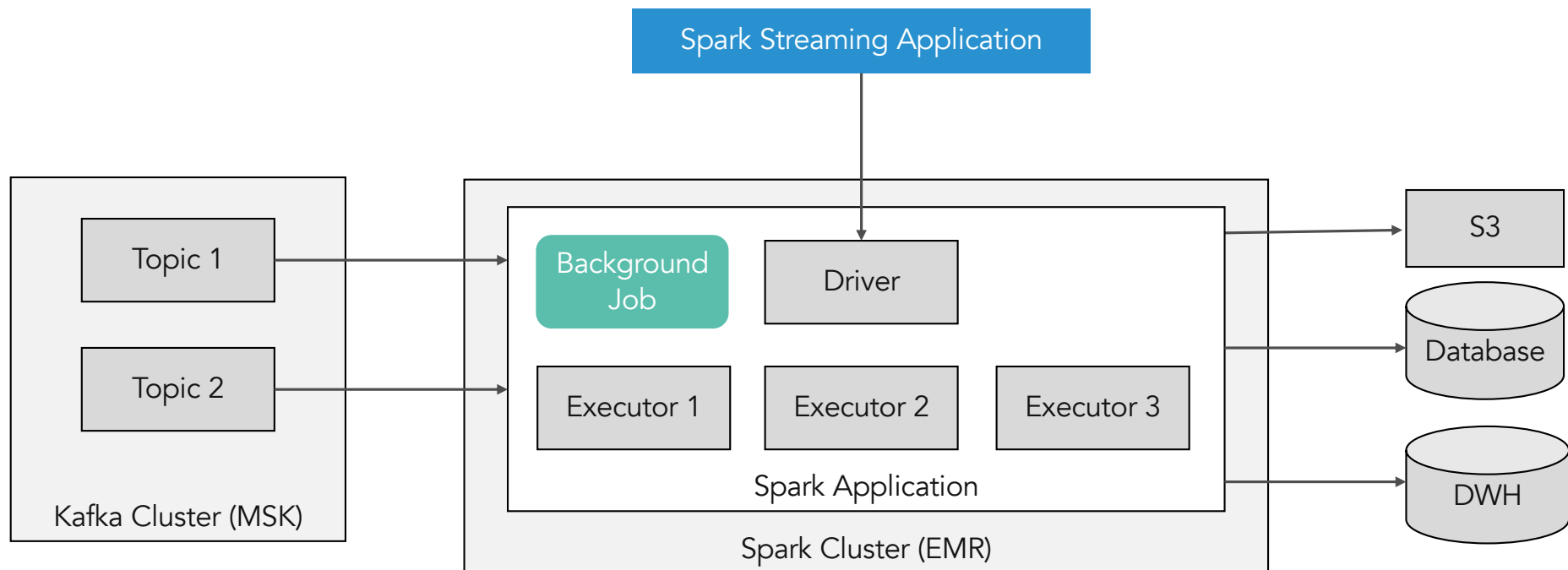
- Kafka sink
- File sink
- Console sink – Display output in console
- Memory sink – Output is stored to a temporary table in memory.
- Database, DWH
- Foreach sink (foreach)
- ForeachBatch sink (foreachPartition)



Processing & APIs

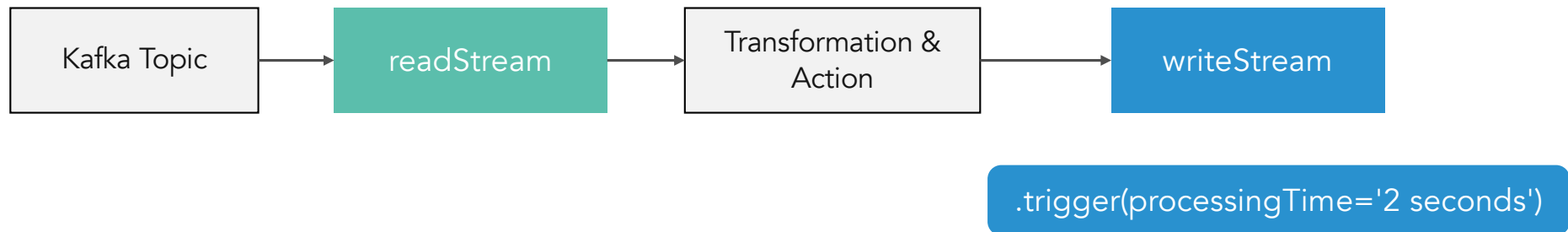
- Transformations & Aggregations
 - Same as batch
- APIs
 - Source/Read – `spark.readStream`
 - Write – `df.writeStream`

Spark Streaming Architecture



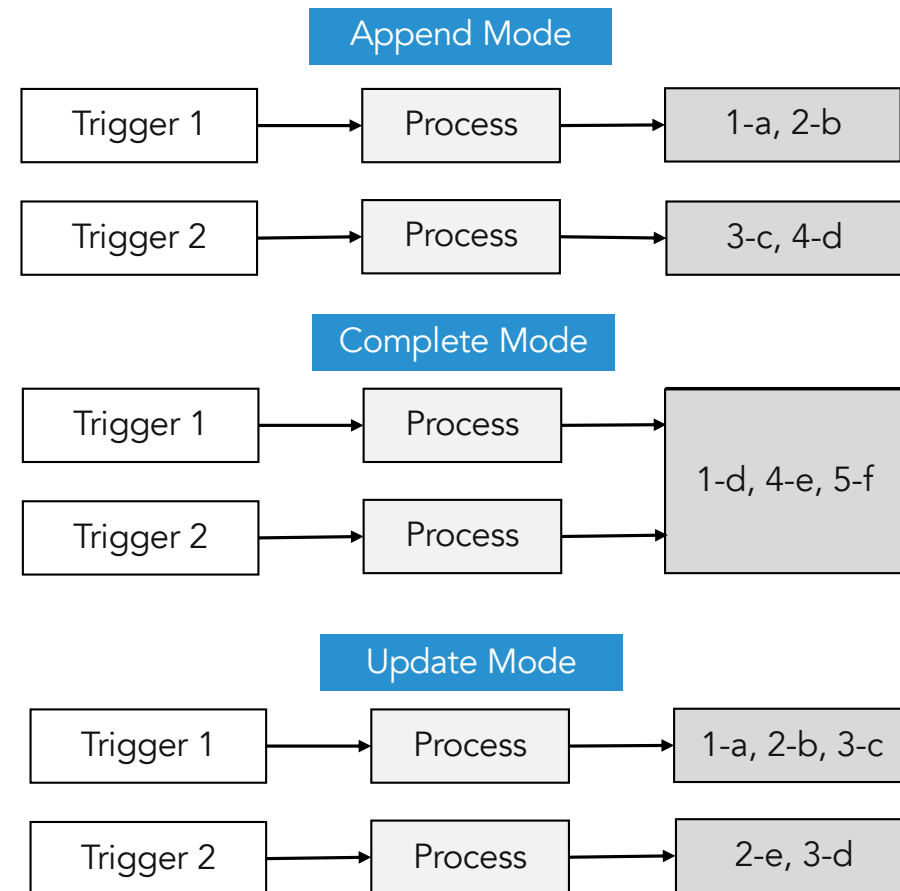
Trigger

- No Trigger.
- Fixed Interval (using Processing Time)
- Available Now
- Continuous



Output Mode

- Append mode. Only supported for queries that will never change the rows that have been added to the output.
- Complete mode. The entire result set is the output after every trigger.
- Update mode. Only rows that have been changed since the last output will be changed.



PySpark Implementation

- Create SparkSession.
- `df1 = spark.readStream.format("kafka")`
- `option("kafka.bootstrap.servers", "localhost:9092")`
- `option("subscribe", "3aayaam-stock-topic").load()`
- `df2 = df1.applyTransformations(...)`
- `df2.writeStream.format("parquet | jdbc | console | kafka")`
- `trigger(processingTime='10 seconds')`
- `option(...).start()`
- `df3.awaitTermination()`

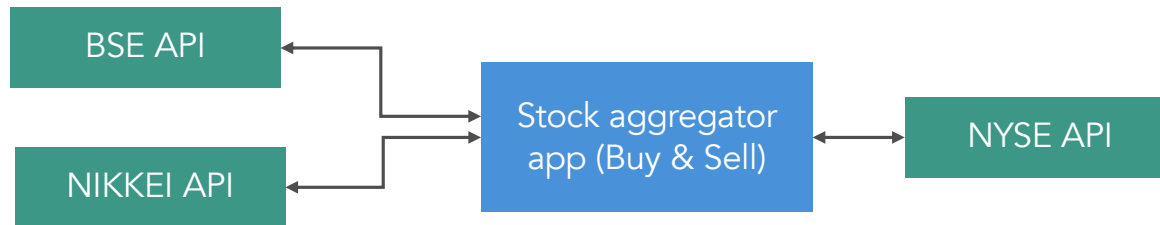
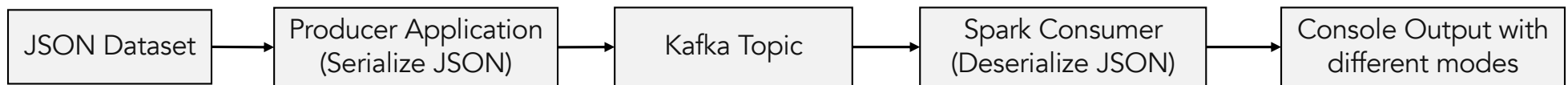
HandsOn

- Create Kafka cluster on laptop.
- Start Spark standalone cluster. Start Kafka cluster.
- Considerations:
 - It is not like reading from CSV, Parquet, database etc.
 - The output would look like below table

key	value	topic	partition	offset	timestamp	timestampType

- Have to extract "value" column from the output.
 - `df.select('value')`
 - `df.select(col('value').cast('string'))`
- Execute with Trigger time of 10 seconds.

HandsOn – JSON SerDe



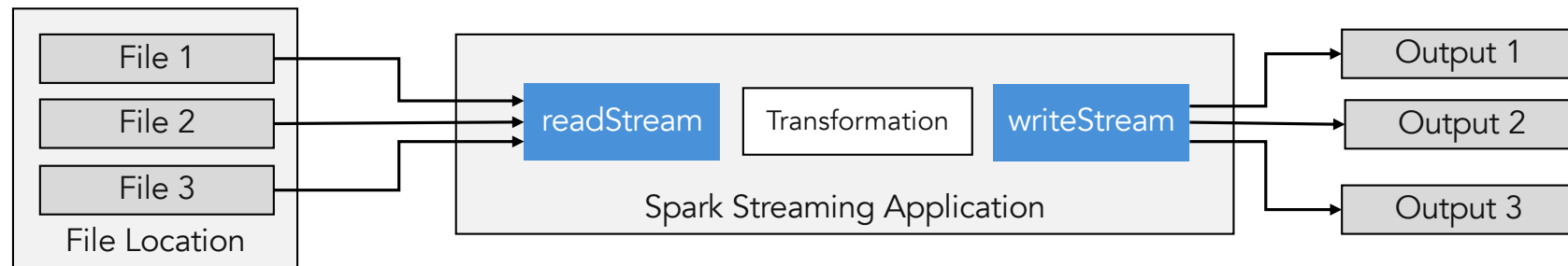
- Stock JSON dataset.
- Create producer to send data to Kafka topic after serializing.
- PySpark streaming code to read from Kafka topic and show in console.
- Use different output modes – APPEND, COMPLETE, UPDATE.

Agenda 2

www.3aayaam.in

1. File Source Triggers
2. State Store
3. Streaming Windows
4. Event Time
5. Watermark

File Sources



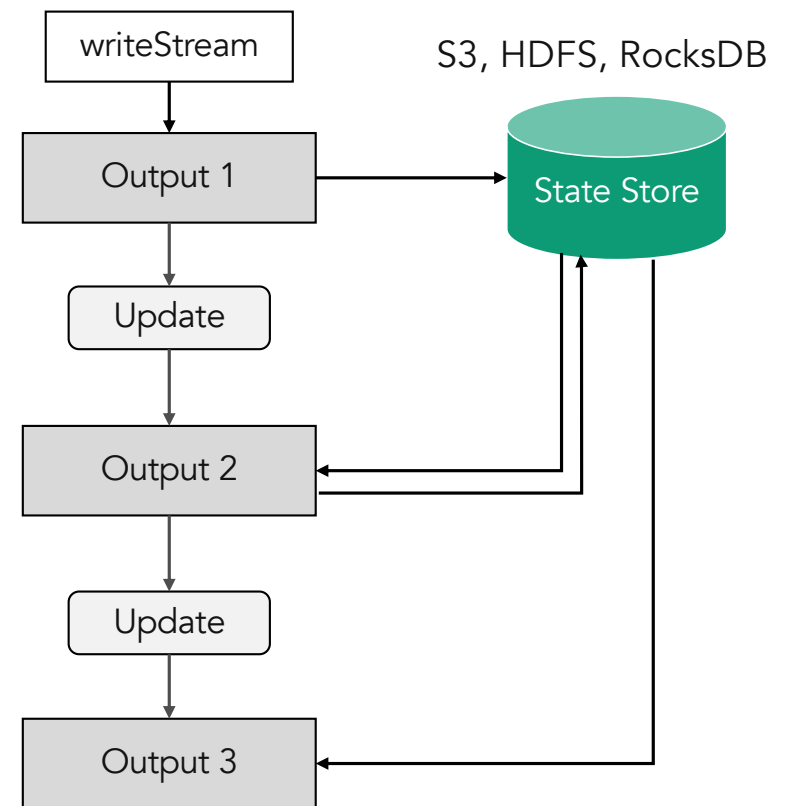
- CSV, JSON, Parquet, ORC
- `readStream.format("JSON").schema(schema).option(<Configuration>).load(<file-location>)`
- Configuration
 - `maxFilesPerTrigger` – Max no. of files to be considered at every trigger.
 - `maxBytesPerTrigger` – Max total size of files to be considered at every trigger.
 - `latestFirst` – Process latest files first.
 - `maxFileAge` – Files older than this will be ignored.
- `spark.sql.streaming.schemaInference`

Unsupported Transformation in Streams

- Count
- Show
- Describe
- Limit
- Distinct
- Sort

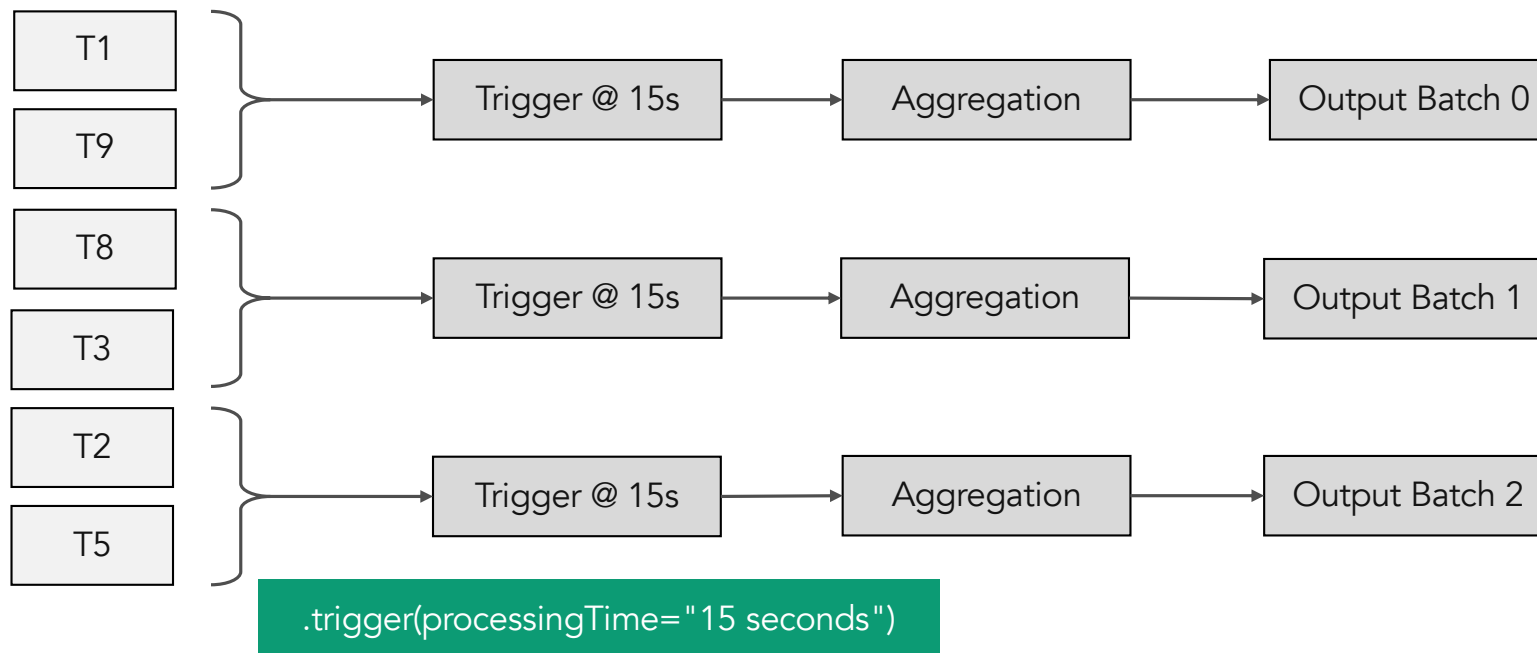
Streaming State Store

- Output Mode – Append, Update, Complete.
- Backed by State Store.
- Used by transformations – Aggregations, Joins (two streams), dropDuplicates, Incremental processing.
- Used with writeStream API.
- `option("checkpointLocation", "<file-location>")`
- Hands-on: State Store
- Hands-on : Output Mode



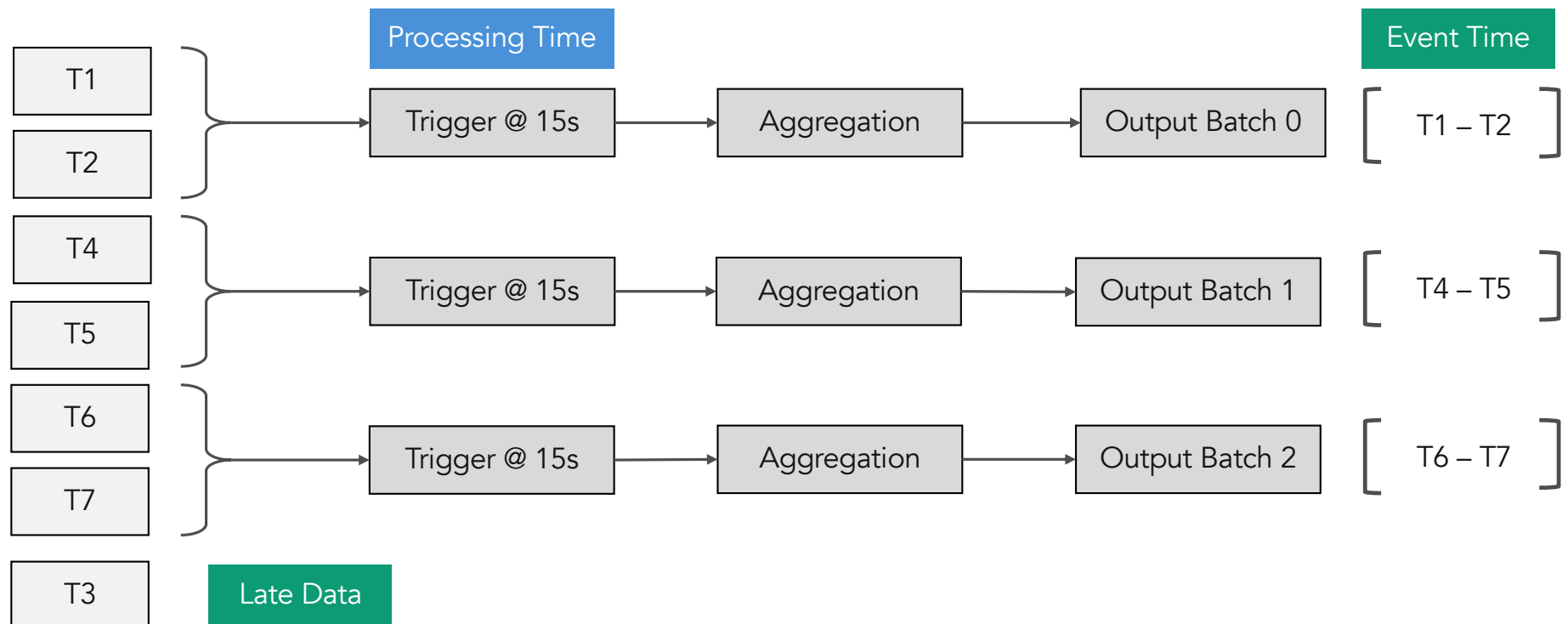
Event Time based processing

- Processing Time – Micro Batch
- Processing time is considered at server (Spark cluster) side.



Event Time based processing

- But this is not how streaming aggregation is used.



Event Time based processing

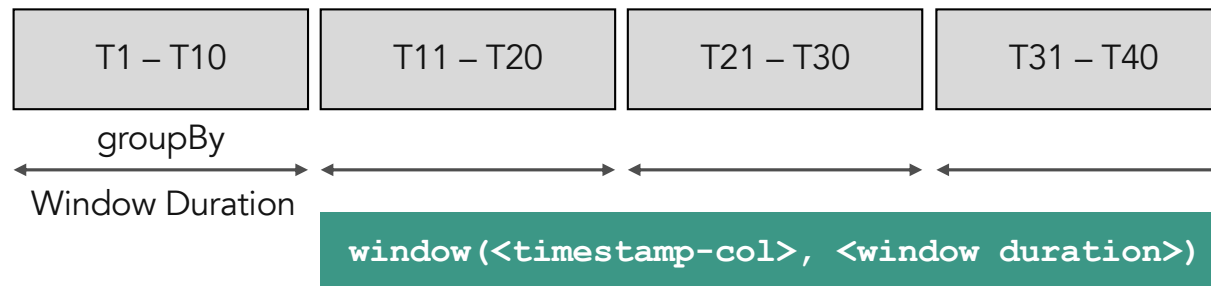
- Event Time is generated at source applications. Associated with individual records.

```
{"trade_id": "YLQ-9467-PF-013", "trade_timestamp": "2025-09-02 07:10:06.623723",  
"trade_direction": "BUY", "country": "France", "symbol": "MLIDS", "exchange": "PSE",  
"operating_hours": 4, "buyer_id": 84457, "seller_id": 81534, "stock_details": {"price":  
"10.83", "volume": 767653, "security_name": "IDS France SAS", "sector": "Technology",  
"currency": "EUR"}}
```

- trade_timestamp – Timestamp generated at application side.
- Processing using Event Time

Event Time Tumbling Windows

- How micro batches should be created?
- What should be the trigger duration?
- Tumbling Window. Based on event time.



- `df1 = ...readStream...`
- `df2 = df1.groupBy(window(col("<evet time col>"), "10 seconds" )).agg(avg())`
- `df2 = df1.groupBy(window(col("<evet time col>"), "10 seconds"), col("col1"), col("col2")).agg()`
- Output Modes will continue to be same.

Tumbling Window Interval

- Hands On.
- Intervals are aligned to the start of second or min or hour.

trade_timestamp	Stock	price
2027-09-03 15:33:10.116889	TATAM	10
2027-09-03 15:35:29.117221	TATAM	10
2027-09-03 15:31:03.117260	TATAM	10
2027-09-03 15:21:28.117285	TATAM	10
2027-09-03 15:24:34.117294	MNM	10
2027-09-03 15:44:24.117302	MNM	10
2027-09-03 15:40:36.117311	TATAM	10
2027-09-03 15:11:37.117319	TATAM	10
2027-09-03 15:16:16.117327	TATAM	10
2027-09-03 15:31:30.117351	MNM	10

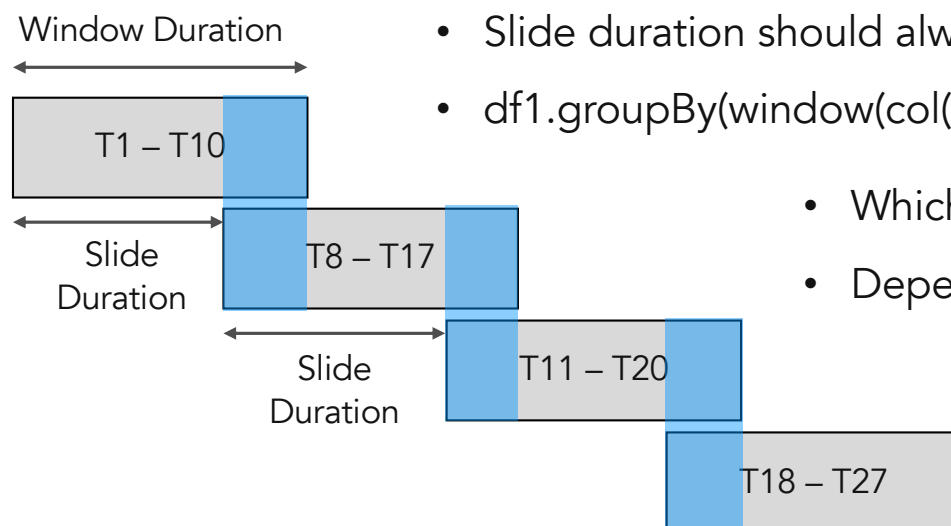
Event Time
Tumbling Window
5 mins

Processing Time
20 seconds

Window	Stocks	Total Price
15:30:00 to 15:35:00	TATAM	10 20
15:35:00 to 15:40:00	TATAM	10
15:30:00 to 15:35:00	MNM	10
15:20:00 to 15:25:00	TATAM	10
15:20:00 to 15:25:00	MNM	10
15:40:00 to 15:45:00	MNM	10
15:40:00 to 15:45:00	TATAM	10
15:10:00 to 15:15:00	TATAM	10
15:15:00 to 15:20:00	TATAM	10

Event Time Sliding Windows

- Sliding Window. Based on event time.



- Slide duration should always be smaller than Window duration.
- `df1.groupBy(window(col("<timestamp-col>"), "10 seconds", "5 seconds"))`
- Which one to use – Tumbling or Sliding window?
- Depends entirely on use case.

```
window(<timestamp-col>, <window duration>, <slide duration>)
```

Sliding Window Interval



trade_times tamp	Stock	Price
15:33:10	TATAM	10
15:35:29	TATAM	10
15:31:03	TATAM	10
15:21:28	TATAM	10
15:24:34	MNM	10
15:44:24	MNM	10
15:34:36	TATAM	10
15:11:37	TATAM	10
15:16:16	TATAM	10
15:31:30	MNM	10

Event Time
Window Duration
5 mins

Sliding Window
Duration
3 mins

Window	Stocks	Total Price
15:30:00 to 15:35:00	TATAM	10 20 30
15:33:00 to 15:38:00	TATAM	10 20
15:35:00 to 15:40:00	TATAM	10
15:38:00 to 15:43:00		
15:20:00 to 15:25:00	TATAM	10
15:23:00 to 15:28:00		
15:20:00 to 15:25:00	MNM	10
15:23:00 to 15:28:00	MNM	10
15:40:00 to 15:45:00	MNM	10
15:43:00 to 15:48:00	MNM	10

State Store

Input

trade_timestamp	Stock	price
2027-09-03 15:33:10.116889	TATAM	10
2027-09-03 15:35:29.117221	TATAM	10
2027-09-03 15:31:03.117260	TATAM	10
2027-09-03 15:21:28.117285	TATAM	10
2027-09-03 15:24:34.117294	MNM	10
2027-09-03 15:44:24.117302	MNM	10
2027-09-03 15:40:36.117311	TATAM	10
2027-09-03 15:11:37.117319	TATAM	10
2027-09-03 15:16:16.117327	TATAM	10
2027-09-03 15:31:30.117351	MNM	10

Output

Window	Stocks	Total Price
15:30:00 to 15:35:00	TATAM	10 20
15:35:00 to 15:40:00	TATAM	10
15:30:00 to 15:35:00	MNM	10
15:20:00 to 15:25:00	TATAM	10
15:20:00 to 15:25:00	MNM	10
15:40:00 to 15:45:00	MNM	10
15:40:00 to 15:45:00	TATAM	10
15:10:00 to 15:15:00	TATAM	10
15:15:00 to 15:20:00	TATAM	10

State Store

Watermark – Introduction

- State Store or Checkpointing.
- Processing time, Event time (Tumbling & Sliding window), Output mode – all use state store for incremental processing.
- Problem 1 – How long should we allow state store to save data.
- Data might come late because of n/w problem, application issue, device breakdown etc.
- Streaming window cannot wait forever.
- Problem 2 – How long a streaming window will wait before late data is considered irrelevant.
- Watermark
 - Late arrival window.
 - Data storage duration in state store.

Watermark – Implementation

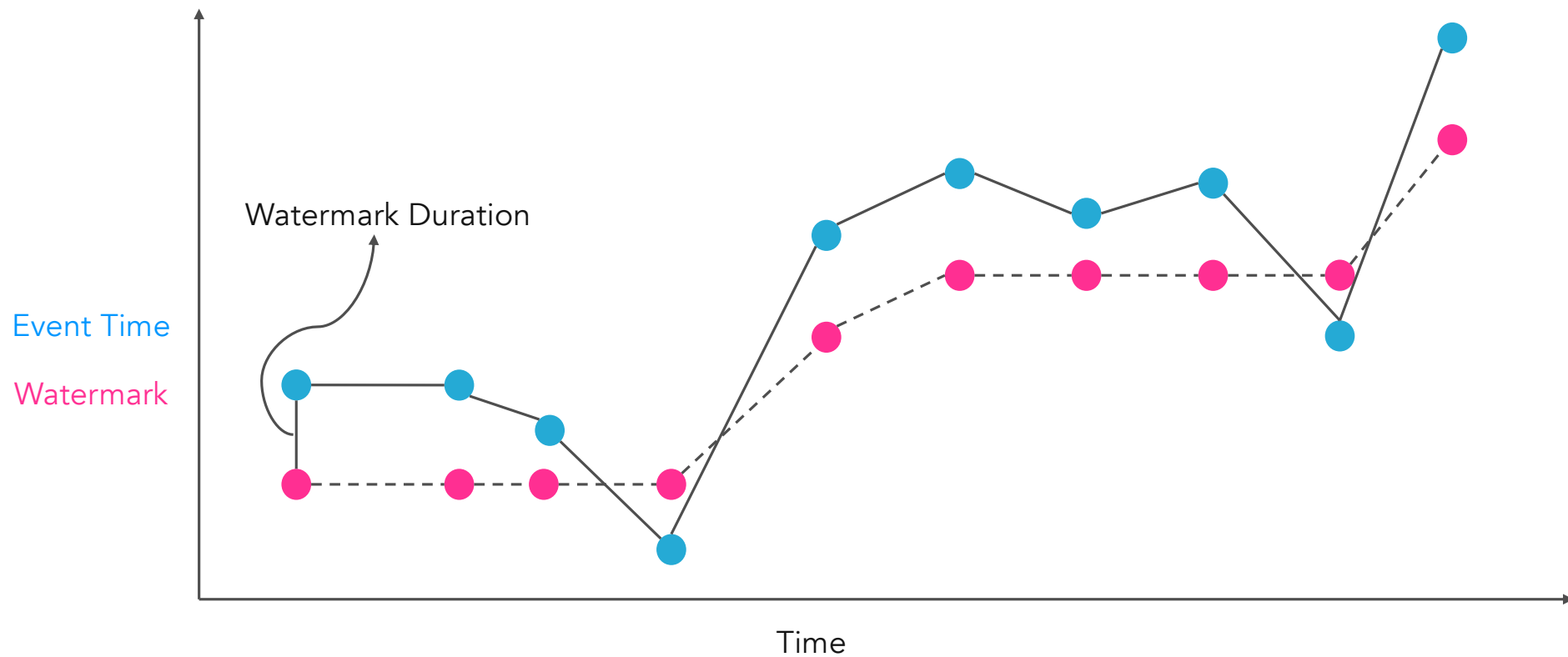
- PySpark Implementation:
 - `df2 = df1.withWatermark("<event-time>", "5 mins").groupBy(window(col("<event-time>"), "10 seconds"), col("col1"))`
 - Discard records that fall in the range older than 5 mins of event time.
 - Event Time = 10.20.00am => discard all records older than range 10.10.5am to 10.15.00am.
- What if the next event time is 10.23am or 10.15am?
- **Watermark = Max(Event Time) – Watermark Duration.**
- It does not depend on window ranges or processing time.

Watermark Example

withWatermark("trade_timestamp", "5 mins")

trade_timestamp	Event Time Window	Watermark	Watermark Window	Discarded
15:33:10	15:30 – 15:35	15:28:10	15:25 – 15:30	NO
15:35:29	15:35 – 15:40	15:30:29	15:30 – 15:35	NO
15:31:03	15:30 – 15:35	15:30:29	15:30 – 15:35	NO
15:21:28	15:20 – 15:25	15:30:29	15:30 – 15:35	YES
15:24:34	15:20 – 15:25	15:30:29	15:30 – 15:35	YES
15:44:24	15:40 – 15:45	15:39:24	15:35 – 15:40	NO
15:34:36	15:30 – 15:35	15:39:24	15:35 – 15:40	YES
15:11:37	15:10 – 15:15	15:39:24	15:35 – 15:40	YES
15:16:16	15:15 – 15:20	15:39:24	15:35 – 15:40	YES
15:31:30	15:30 – 15:35	15:39:24	15:35 – 15:40	YES

Watermark Graph



Watermark & Output Modes

- APPEND - Used for queries without aggregation.
 - CANNOT be used with Watermark.
- COMPLETE – Replace the entire result set after recalculating aggregation.
 - CAN be used with Watermark. But used only to manage state store. Late arrival data is not discarded.
- UPDATE – Writes modified and new records, with aggregation, in the stream to the sink.
 - CAN be used with Watermark. Watermark works as state store management and discarding late data arrival.
- Hands On

Deduplication

- Drops duplicate data with and without Watermark.
- `dropDuplicates( [ "col1", "col2" ] )`
- `withWatermark("<event-time>", "duration").dropDuplicates( [ "col1", "col2" ] )`
- `dropDuplicatesWithinWatermark( [ "col1", "col2" ] )`

Summary

www.3aayaam.in

- Streaming Ingestion & Structured Streaming
- Streaming Architecture
- Streaming Output Modes
- File Source Triggers
- State Store
- Event Time Processing
- Watermark

A glass globe with a forest scene inside, resting on moss with a small plant growing next to it.

Thank You
www.3aayaam.in